NeuraEdge Universal
Vivado Build, Schematic, Waveform, Timing, and Power Guide

Exact flow for Vivado 2026.1 and PYNQ-Z2

July 16, 2026

Contents

1 Installed paths used by this repository **2**

2 Run verification before implementation **2**

3 Validate the complete PYNQ block design **2**

4 Build the full bitstream **2**

5 Open the exact implemented design in Vivado **3**

6 Generate and inspect schematics **3**

7 Run a waveform simulation in the Vivado GUI **3**

7.1 Create a simulation-only project 3

7.2 Recommended waveform groups 4

7.3 Radix and analog display 4

7.4 Capture every repository waveform automatically 4

7.5 How to read a valid/ready waveform 4

8 Save a reusable waveform configuration **5**

9 Timing analysis **5**

10 Activity-based power analysis **5**

11 Run one exported image on PYNQ **5**

12 Failure diagnosis checklist **6**

13 Vendor references **6**

1 Installed paths used by this repository

The scripts expect Vivado at `D:\AMDDesignTools\2026.1\Vivado`. The target is PYNQ-Z2 part `xc7z020clg400-1`. The project-local board definition is under `third_party/XilinxBoardStore`; no Windows service or startup task is required. The complete installation/removal ledger is `TOOLS_INSTALLED_AND_REMOVAL.txt`.

Open PowerShell in `neuraedge_universal`. Use PowerShell 7 or Windows PowerShell; commands below are repository-relative.

2 Run verification before implementation

```
.\scripts\run_regression.ps1
```

The command regenerates independent fixtures, runs strict Verilator lint, executes all Python tests, compiles all RTL with Icarus, and runs all 14 testbenches. A successful final line is `NEURAEDGE_REGRESSION_PASSED`. Logs are retained under `outputs/regression`. Do not proceed from a modified RTL snapshot if this gate fails.

3 Validate the complete PYNQ block design

```
.\scripts\run_pynq_full.ps1 -Mode validate
```

Validation creates the processing-system, DMA, three clock/reset islands, two AXI-Lite clock converters, two AXI-Stream clock converters, interconnect, DDR, and accelerator IP connections and runs Vivado block-design validation. It is faster than implementation and catches address, clock, interface, and IP-parameter errors. Confirm the DMA IP reports `c_sg_length_width=26`. The intended domains are 100 MHz control, 125 MHz DMA/HP0, and 200 MHz accelerator.

4 Build the full bitstream

```
.\scripts\run_pynq_full.ps1 -Mode full -CleanLogs
```

This command performs synthesis, optimization, placement, physical optimization, routing, DRC, setup/hold analysis, utilization, power, and bitstream export. It also exports HWH/XSA handoff data for PYNQ software. Keep the PowerShell window open until the command returns; implementation is not finished merely because synthesis reaches 100%.

Artifact	Meaning
`outputs/pynq_full/neuraedge_pynq_full.bit`	FPGA configuration bitstream.
`outputs/pynq_full/neuraedge_pynq_full.hwh`	PYNQ hardware metadata and register/interface description.
`outputs/pynq_full/neuraedge_pynq_full.xsa`	Vivado hardware handoff archive.
`full_routed_timing_summary.rpt`	Setup/hold timing after physical routing.
`full_routed_timing_paths_250.rpt`	250 detailed routed timing paths.
`full_routed_utilization_hierarchical.rpt`	LUT, FF, BRAM, DSP, and hierarchy utilization.
`full_drc.rpt`, `full_cdc.rpt`, `full_methodology.rpt`	Unfiltered implementation checks.
`full_vectorless_power.rpt`	Vivado default-activity power estimate and confidence details.
`neuraedge_pynq_full_routed.dcp`	Reopenable implemented design checkpoint.
`neuraedge_pynq_block_design.pdf/.svg`	Native Vivado `system.bd` exports.

Artifact	Meaning
`D:/NeuraEdgeBuild/pynq_full/*.xpr`	GUI project containing the exact block design and runs.

5 Open the exact implemented design in Vivado

Start Vivado from the Start menu or run:

```
& 'D:\AMDDesignTools\2026.1\Vivado\bin\vivado.bat'
```

Choose **Open Project** and select `D:/NeuraEdgeBuild/pynq_full/neuraedge_pynq_full.xpr`. This short generated path avoids the Windows 260-character limit in nested IP files. Set `NEURAEDGE_VIVADO_BUILD_ROOT` before a build if a different cache location is required. In the Flow Navigator:

1. Open **IP Integrator** > **Open Block Design** to inspect PS, DMA, interconnects, clock/reset, and accelerator connectivity.
2. Choose **Open Implemented Design** before reading timing, utilization, placement, or routed schematics. "Open Synthesized Design" does not include final route delay.
3. Open **Reports** > **Timing** > **Report Timing Summary**. Setup WNS and hold WHS must both be non-negative for every reported clock. The retained final report reads +0.005 ns setup WNS and +0.050 ns hold WHS.
4. Open **Window** > **Device** to see placement and routing on the physical Zynq-7020 fabric.

6 Generate and inspect schematics

The build script automatically launches the required Vivado GUI-mode export after batch implementation because `write_bd_layout` is unavailable in batch mode. Open `outputs/pynq_full/neuraedge_pynq_block_design.pdf` or the SVG; both come directly from the validated `system.bd`. The expected path is:

PS DDR ↔ 125 MHz HP0 AXI ↔ AXI DMA ↔ stream CDC ↔ 200 MHz NeuraEdge, with 100 MHz PS GP0/AXI-Lite ↔ control CDC ↔ the accelerator.

For RTL structure, select `system_i/accelerator/inst` in the hierarchy, then choose **Schematic**. Expand only one level at a time. The expected accelerator chain is PYNQ AXI top → descriptor sequencer and universal core → buffered tensor/vector/normalization/LIF/LUT/SGD engines. Expanding the complete tensor array at once creates an unreadable schematic because 64 DSP blocks and their bank wiring are intentionally parallel.

7 Run a waveform simulation in the Vivado GUI

7.1 Create a simulation-only project

In Vivado choose **Create Project**, select **RTL Project**, and do not specify sources initially. Set the part to `xc7z020clg400-1`.

Add all `rtl/*.sv` files as Design Sources. Ensure `neuraedge_command_pkg.sv` is compiled before modules that import the package. Add one `tb/tb_*.sv` file as a Simulation Source. Under **Simulation Settings**, set that testbench module as the simulation top. The testbench clock uses a 2.5 ns half-period, which represents 200 MHz.

Choose **Run Simulation** > **Run Behavioral Simulation**. In the Scope window select `dut`; in Objects select signals and choose **Add to Wave Window**. Use **Restart**, then **Run All**.

7.2 Recommended waveform groups

Testbench	Signals and the question they answer
`tb_neuraedge_descriptor_sequencer`	`state`, program active/done/error, all input/output valid-ready-data-last, command valid/ready/done, remaining counters, output-buffer valid. This proves record parsing, backpressure, output TLAST, and overrun recovery.
`tb_neuraedge_pynq_descriptor`	AXI-Lite AW/W/B and AR/R channels, descriptor start, streams, interrupt, status, record count. This connects software-visible registers to descriptor execution.
`tb_neuraedge_universal_core`	command opcode/valid/ready, selected-engine state, input/output stream, busy, done, error, dense-MAC/spike-add/update/spike counters. This shows opcode dispatch and engine ownership.
`tb_neuraedge_buffered_tensor_core`	load state, item/segment indices, active-index write/count, issue FIFO, tensor start/busy/result. This shows tile loading, zero compaction, and compute.
`tb_neuraedge_tensor_tile`	mode, issue valid/ready, lane activation/weight, accumulator valid, result row and column, dense/spike metrics. Display binary/ternary values in signed radix.
`tb_neuraedge_lif_bank`	lane index, current, old/leaked/integrated membrane, threshold, spike output, writeback, timestep and segmented spike-count states.
`tb_neuraedge_sgd_optimizer8`	weight, gradient, velocity, momentum product, learning-rate product, rounded delta, saturation, write address, result stream, update count.
`tb_neuraedge_normalization_units`	LayerNorm state, sum, squared sum, mean, variance numerator, square-root start/done, divider start/done, gamma/beta, output; then softmax maximum, differences, exponents, denominator, output.

7.3 Radix and analog display

Use unsigned decimal for counts and addresses. Use signed decimal for membrane, weights, gradients, accumulators, gamma, beta, and normalized values. Use hex for 64/128-bit stream words because command fields are easier to align by nibble. Use binary for one-bit handshakes and spike vectors. Vivado's analog wave style is useful for membrane trajectories and accumulators, but retain the digital value column so exact fixed-point values remain visible.

7.4 Capture every repository waveform automatically

The reproducible command-line capture uses the same Vivado XSim engine:

```
.\scripts\capture_waveforms.ps1 `
  -OutputDirectory .\outputs\waveforms_final
```

It produces 15 VCD files for portable inspection, 15 WDB files retaining Vivado-only arrays and types, simulation logs, and a byte-counted JSON manifest. GTKWave can open a VCD; Vivado can open the corresponding WDB. VCD cannot represent unpacked SystemVerilog arrays, so use WDB when an internal memory or lane array is absent from GTKWave.

7.5 How to read a valid/ready waveform

Place valid, ready, data, and last adjacent in that order. Add a divider after each interface. A beat transfers only at a rising edge with valid and ready high. During a stall, data and last must not change. On the final accepted input beat, payload TLAST must agree with the command's declared input length. On the final global output beat, DMA TLAST must be one even if the final descriptor record itself has no output.

8 Save a reusable waveform configuration

After arranging signals, dividers, colors, and radix, choose **File > Save Waveform Configuration** and store a `.wcfg` file under `docs/waveforms`. A WCFG stores presentation only; it does not replace the WDB/VCD event data. Re-run simulation after RTL changes, then reopen the WCFG so signal grouping remains consistent.

9 Timing analysis

Read the routed report, not the synthesis estimate. For setup timing:

$$\text{slack} = \text{required arrival time} - \text{actual arrival time}.$$

Negative setup slack means data reaches a destination register too late for the next edge. Inspect source register, destination register, logic delay, route delay, and logic levels. A route-dominated path usually needs a register boundary, localized control, reduced fanout, or better physical locality; merely rewriting an equivalent Boolean expression may not solve it.

Hold timing asks whether data changes too soon after the same clock edge. Hold WHS must also be non-negative. Never quote a frequency from a design with a negative setup or hold result. The OOC build constrains top-level I/O delays and is useful for core iteration; the full block design is the board integration signoff because its clocks, interconnect, and DMA are physically present.

10 Activity-based power analysis

Vivado vectorless power assumes switching rates for unannotated nets. For workload-informed power, simulate a representative command stream in XSim, record SAIF over a clearly stated time window, open the routed design, apply `read_saif` to the matching hierarchy, and run `report_power`. Check the report's SAIF annotation percentage and confidence. Do not rename vectorless power as measured board power.

The exact full-checkpoint rerun command is:

```
.\scripts\run_saif_power.ps1 `
  -Checkpoint .\outputs\pynq_full\neuraedge_pynq_full_routed.dcp `
  -OutputDirectory .\outputs\power_activity_full `
  -SaifDirectory .\outputs\power_activity_full `
  -AcceleratorInstance system_i/accelerator/inst
```

The Tcl flow losslessly relocates only the two simulated SAIF wrapper-instance names to the implemented accelerator hierarchy, imports activity with `-no_strip`, and deletes the temporary relocated files. The retained report matches 9,164 of 123,250 full-design nets and estimates 1.937 W total, 1.785 W dynamic, and 0.472 W accelerator-hierarchy dynamic power at medium confidence. Untouched memory creates explicit X-state warnings; retain these when quoting the estimate.

For physical energy, connect a PYNQ-Z2, measure the relevant supply rail with a calibrated instrument, subtract an explicitly measured idle baseline when reporting accelerator dynamic power, and integrate power over hardware elapsed time. Record bitstream hash, clock, sample count, board revision, supply voltage, instrument, sampling rate, and ambient conditions.

11 Run one exported image on PYNQ

Copy the generated BIT/HWH and parameter directory onto the board or mount the repository, then run:

```
python software/run_spikingformer_pynq.py \
  --bitfile outputs/pynq_full/neuraedge_pynq_full.bit \
  --manifest outputs/spikingformer_hardware_parameters/spikingformer_neuraedge_weights.json \
```

```
  --input sample.npy \
  --output result.json
```

The result JSON records integer accumulators, dequantized logits, prediction, elapsed time, throughput, and hardware telemetry. The receive DMA is armed before submission/send to prevent output backpressure from deadlocking a command that produces results before all host-side calls have returned.

12 Failure diagnosis checklist

Symptom	Check
No input ready	Program-start bit, descriptor mode, reset, sequencer state, header/input lengths, and whether the selected core engine is busy.
No output	Arm S2MM receive first; check declared output count, command opcode, core done/error, output-buffer valid, and DMA status.
Program error	Compare record input/output counts with compiler output; inspect TLAST at command-high/payload end; read completed-record counter.
Simulation hangs	Locate the last handshake, inspect ready backpressure, and verify the testbench watchdog message rather than extending timeout blindly.
Negative setup slack	Open the worst routed path and identify the actual cross-module/fanout dependency; add a verified register boundary where protocol semantics permit one cycle of latency.
Power confidence low	Increase SAIF hierarchy match and simulation coverage; state unmatched-net assumptions in the report.
Python rejects parameters	Do not disable checks. Re-export the archive and compare whole-file and section hashes to the manifest.

13 Vendor references

- AXI DMA PG021: `https://docs.amd.com/r/en-US/pg021_axi_dma/Field-Descriptions`
- Vivado power analysis UG907
- Vivado Design Suite user guides: `https://docs.amd.com/`